

DATA-STRUCTURES & ALGORITHMS

with

Manoj Prabhakaran

Lecture 8

Double-Ended Queues

And Amortisation

Deque

A feature of `std::deque`, which makes it more versatile

- What is a deque?
 - Can add items to either end of a deque (`push_back`, `push_front`)
 - Can remove from either end too (`pop_back`, `pop_front`)
 - Can read from either end (`front()`, `back()`)
 - **Random access:** Can read the i^{th} item (0^{th} being front) (using `[i]` or `at(i)`)
- Why are deques useful in programs?
 - Used in C++ STL to implement stacks and queues! A deque has all the methods needed by either (and also by vectors)
 - Deque can implement a forgetful stack. Or more custom versions where forgetting depends on the element (not based on the capacity)
 - Systematic searches using a container to hold items to explore, allowing more flexibility, still with $O(1)$ time insertions/deletions

Deque

- Can implement using a linked list, and have $O(1)$ {push/pop}_{front/back}
 - Can we get better constants?
- As a list of fixed-capacity dequeues (dequeBlock)?
 - A circular array can give a dequeBlock (easy extension of our queueBlock from the last time)
- Almost works, but **doesn't support random access!**
- First idea (we will improve on this): Reallocate the dequeBlock with double the capacity, each time we outgrow it
 - $O(N)$ worst-case but $O(1)$ amortised cost

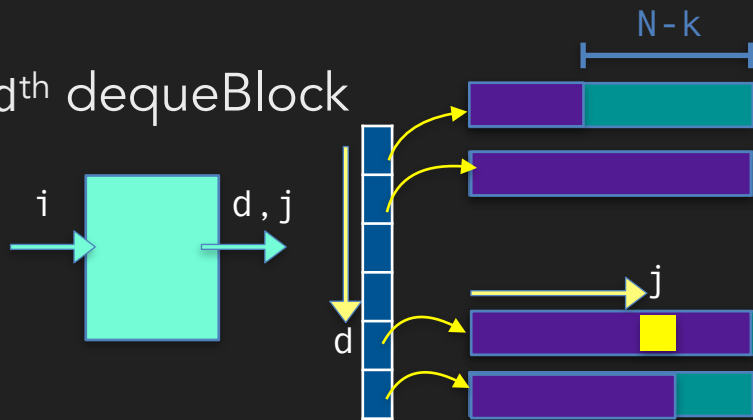
Amortised Cost

Important that we increase the size by a multiplicative factor

- Let us consider the reallocation cost of `std::vector` (same holds for our `dequeBlock`)
- Reallocation strategy: On reaching size M , we will reallocate the vector to have size $2M$, and copy the M data items to the new location
 - We paid a reallocation cost of M after at least $M/2$ inserts since the last reallocation
 - Account this cost as spread across those $M/2$ inserts: each insert is charged an additional cost of $2 = O(1)$ moves
 - Important: An insert is charged only once — for the next reallocation
 - On average, the cost is still $O(1)$!

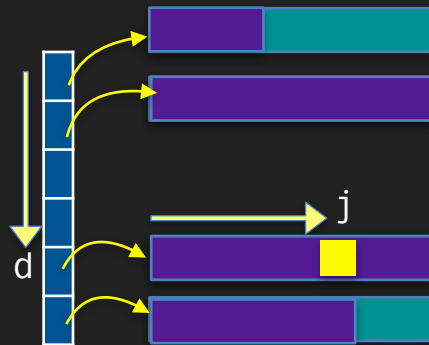
Deque

- For real-time systems, amortised guarantees are often not good enough
- Can we mitigate the $O(N)$ worst-case cost? Maybe reduce the constant?
- A list of dequeBlocks almost worked, but didn't support random access
- What do we need for random access?
 - Translate an index i to (d, j) such that the item is at index j in the d^{th} dequeBlock
- Use a "directory" to get a pointer to the d^{th} dequeBlock
- If the first dequeBlock has k elements,
 $d = (i + N - k) / N$, and $j = (i + N - k) \% N$
(unless $i < k$, in which case $j = i$)



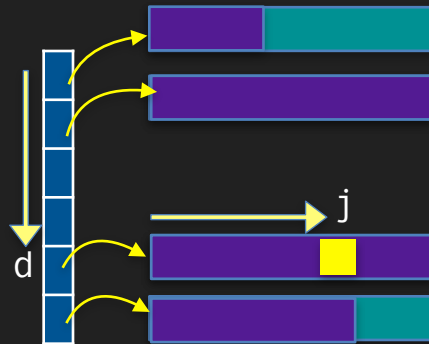
Deque

- `directory[d]` should have the pointer to the d^{th} dequeBlock
- Should allow adding/removing dequeBlock entries to either end of the directory
 - We need a deque for the directory! But we are trying to build one...
 - Use a dequeBlock? But it will fill up...
 - We will reallocate when that happens! Not very expensive now!
 - Recall from our `std::vector` analysis: We spend $O(M)$ time for every M entries in the directory
 - That is once for $N = M \times B$ items in the deque
 - $O(1/B)$ amortised, $O(N/B)$ worst-case cost



Deque: Exercise

- `std::deque` (just like `std::vector`) allows inserting and deleting from anywhere, in $O(N)$ time
- We can actually guarantee $O(B) + O(N/B)$ time
 - Which is technically still $O(N)$ if you consider B a constant. But it is a large constant in practice, and can be modelled as a separate growing parameter.
- Idea: To insert/erase within a dequeBlock, we take time $O(B)$. Then in a sequence of pop/push operations, update the $O(N/B)$ dequeBlocks to one side of the index that is erased/inserted
- Exercise: do it!

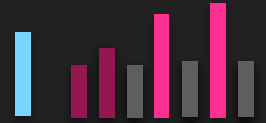


An Application: Max in Window

- Finding the max in a window in $O(1)$ time
- Given a streaming sequence of numbers x_i , at any time t , find
$$m_t = \max\{x_t, x_{t-1}, \dots, x_{t-N+1}\}$$
 - Need to store many items from the stream: an x_i that is not the maximum in the current window can later become the maximum in a future window (after the current maximum drops out)
- Naive solution: Maintain the N -long window as a queue, and each time scan it to find the maximum. Total time after T inputs: $O(NT)$
 - Can we do it in time $O(T)$ instead?

An Application: Max in Window

- Idea: Maintain the set of elements within the window that are eligible to be selected as max in the future
 - When a new element x_t arrives, all the elements in the set which are $\leq x_t$ can be removed
 - Because they will never get a chance to be the max again as this element will be in the window longer than they are
 - Note: Some elements would have been removed by previous items
- Refined idea: Maintain it as a stack of monotonically decreasing elements
 - When an item arrives, the elements less than it are at the top of the stack; pop them and then push the new item



An Application: Max in Window

- Max element at any point is at the bottom of the stack
 - Will use a *deque* to access it
- How about items which get older than the window?
 - The queue is also sorted by arrival time
 - Remove items outside the window from the other end
 - Supported in a deque



An Application: Max in Window

```
#include <deque>
```

```
struct item {int val; unsigned int time; };
```

```
struct winMax {
```

```
    unsigned int now = 0; const int N=50;
```

```
    std::deque<item> window; // max candidates from last N items
```

```
    void ingest(int x) {
```

```
        // first remove at most one item that falls outside the window
```

```
        if(now>0 && now - window.front().time == N) window.pop_front();
```

```
        item xi{x,now++}; // prepare the new item, and increment time
```

```
        // remove all items superseded by the current item
```

```
        while(!window.empty() && window.back().val < x) window.pop_back();
```

```
        window.push_back(xi); // window always holds the newest item
```

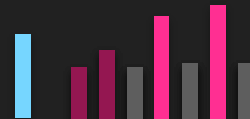
```
    }
```

```
    bool getMax(item& m) { // window is sorted, max is always at the front
```

```
        if(now==0) return false; m = window.front(); return true;
```

```
    }
```

```
};
```



An Application: Max in Window

- How much time does this take?
- When an item is pushed into the stack, need to pop all smaller items, and this can take $\Theta(N)$ time in the worst-case
 - Since T such items, $O(NT)$
 - That is no better than our naive approach!
- But wait, we can count differently!
 - Each item is pushed in exactly once and popped out at most once (from either end of the deque). $\Theta(T)$ push/pop operations in all!
- Processing a single item can take $\Theta(N)$ time in the worst case, but that won't happen for every item. Total is $O(T)$.

Amortised $O(1)$ time per item